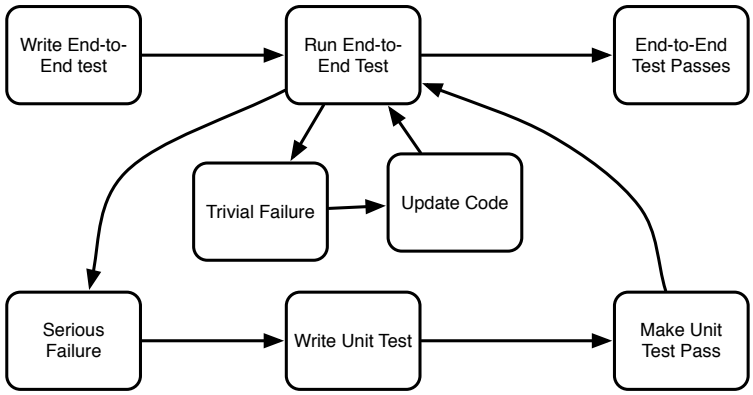
The process is illustrated in the figure that follows—I'm assuming the creation of a new user-facing feature in a Rails application.



Let's walk through the steps.

1. Write the end-to-end test that shows a user interacting with the new feature. This test should specify data, have one or more user interactions, and validate HTML responses to determine that the interactions behave as expected. This should be the main, error-free, interaction of the feature. Before this process even starts, it's perfectly normal to noodle around with controllers and views trying to figure out exactly how the user interactions look. You may even get a substantive part of the feature done. You need to be willing to rewrite that code once the tests come into play.

2. Run the tests. In a reasonably mature Rails application, the first few steps will often already pass. You'll frequently be adding a new feature to an existing page, so steps where data is prepared, users are logged in, and existing pages are hit might all work.

3. Come to a test failure. The first failures you see in integration testing are often trivial: attempts to click on links that don't exist and form fields that aren't there yet. These can be quickly dispatched, generally without unit tests, on the theory that adding new stuff to a view isn't normally testable logic.

4. Cross the chasm. You'll get to the point where you're validating responses and there's a lot of logic missing. Now you have a focused, well-defined task to accomplish in your code, and it's time to drop to unit tests.